AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

i-CREDIT HOURS ENGINEERING PROGRAMS

Deep Learning (CSE 485/488) - Final Project - Milestone 1 report

From Pixels to Language: Bird Classification and Encoder Training on CUB-200-2011

Submitted by Team 7:

1. Hla Waleed 23P0210
2. Malak Khaled 2300447
3. Kenzy Tarek 2301141

Table of Contents

1. Introduction 3
   1.1 Project resources 4
2. Problem Definition 4
3. Dataset Description 5
4. Data Understanding and EDA 5
   4.1 Sample images 6
   4.2 Class distribution 6
   4.3 Image sizes and quality 7
   4.4 Caption statistics 8
   4.5 Summary of anomalies 9
5. Data Cleaning, Preprocessing and Augmentation 10
   5.1 Caption cleaning 10
   5.2 Image preprocessing 11
   5.3 Augmentation strategy 11
   5.4 Colorization data (planned for Milestone 2) 12
6. Models 12
   6.1 ScratchCNN (baseline trained from scratch) 12
   6.2 Pretrained encoders 13
   6.3 Training variants 13
7. Experiments 13
   7.1 Training setup 13
   7.2 Results on the test set 14
   7.3 What the comparison shows 15
8. Hyperparameter Tuning 16
   8.1 Interpretation 17
   8.2 Does tuning carry over to the full run? 18
9. Experiment Tracking 18
10. Evaluation Metrics 21

11. Results and Discussion 22
12. Explainability: Grad-CAM 23
12.1 ResNet-50 fine-tuned (best model) 24
12.2 ScratchCNN (baseline trained from scratch) 26
12.3 What the two models tell us 27
13. Benchmarking 28
14. Final Model Selection 29
15. Failures and Limitations 30
15.1 What went wrong 30
15.2 Limitations 31
15.3 What we would try next 32
16. Team Contributions 32
17. Conclusion 32
18. References 33

1. Introduction

This project builds one convolutional encoder and reuses it for three tasks on the same data: image classification, image colorization and image captioning. The work is split into two milestones. **This report covers Milestone 1**: exploratory data analysis, cleaning and augmentation (Task 1.1), and classification with encoder training, transfer learning, hyperparameter tuning, experiment tracking and Grad-CAM explanations (Task 1.2). Colorization and captioning are Milestone 2 (due 18 December) and are only planned here, not implemented.

We chose **CUB-200-2011**, a fine-grained dataset of 200 bird species, in the version with captions published on Hugging Face (`Multimodal-Fatima/CUB_train` and `CUB_test`). It suits the whole project: the species labels give a hard classification problem where the differences between classes are small, every image comes with ten human-written captions

for the captioning task, and the images are in colour, so grayscale/colour pairs for colorization can be built from the same images.

The questions this milestone answers are:

- How good is a small CNN trained from scratch on about 4,800 training images, and how much does transfer learning from ImageNet add?
- Which pretrained encoder (ResNet-18, ResNet-50, EfficientNet-B0) is best, and is it better to freeze it or fine-tune it?
- Do Grid Search and Random Search find better hyperparameters, and does that carry over to a full-length training run?
- Where does the best model look when it is right and when it is wrong (Grad-CAM), and how far is it from published results?

The code is pure Python/PyTorch, run from a command line (`python -m src.cli train|tune|evaluate|gradcam|predict|figures`), seeded with 42, tracked in Weights & Biases, and reproducible from `README.md` and `requirements.txt`. Models were trained on a free Colab T4 GPU.

1.1 Project resources

Table 1. Where to find the code, models and experiment logs (access is limited to the team and the teaching staff).

Resource	Link
Source code (GitHub, full commit history)	https://github.com/hla06/From-Pixels-to-Language
Trained models, results and figures (Google Drive)	https://drive.google.com/drive/folders/10oDTZjIkUaUYYdQJh_dtkMqeObfuFMyq
Experiment tracking (Weights & Biases)	https://forge.coreweave.com/wandb/hla-ain-shams/pixels-to-language
Colab runner notebook	https://colab.research.google.com/github/hla06/From-Pixels-to-Language/blob/main/notebooks/colab_runner.ipynb
Setup and run commands	`README.md` in the repository

2. Problem Definition

Input: one RGB photograph of a bird (variable size, at most 500 px on a side). **Output:** one of 200 species labels. The encoder that produces the label also produces a feature vector (512, 2048 or 1280 values depending on the architecture); that vector is what the colorization and captioning models will reuse in Milestone 2.

The problem is hard for three reasons. The classes are visually close (several terns, gulls, sparrows, warblers and shrikes differ only in small marks). Each class has only about 30

training images. And the birds appear at different scales, poses and backgrounds. Random guessing scores 0.5% (1 in 200).

Milestone 1 tasks (from the project specification):

- **Task 1.1** Data understanding and EDA: class distribution, image sizes and quality, caption statistics, anomalies; cleaning, splitting and an augmentation strategy.
- **Task 1.2** Train a CNN encoder from scratch, apply transfer learning with pretrained models and compare them, tune hyperparameters, track the experiments, and explain predictions with Grad-CAM.

3. Dataset Description

The dataset is the Hugging Face release of CUB-200-2011 with captions. The provided train and test splits were kept. The test set is not used for training, early stopping, hyperparameter tuning or choosing the final model (all of those use the validation split); it only reports final scores.

Table 2. Dataset summary.

Property	Value
Source	`Multimodal-Fatima/CUB_train` (train) and `Multimodal-Fatima/CUB_test` (test)
Classes	200 bird species
Train images	5,994
Test images	5,794
Columns	`image`, `description`, `label`, `file_name`, `id`
Captions	`description` is one string holding ten captions joined by newlines
Image size	up to 500 px on the longer side, varying aspect ratio

Validation split. The provided data has no validation set, so we carved one out of the train split: 20% of the images, stratified by class with a fixed seed (42), giving **4,795 training, 1,199 validation and 5,794 test images**. Validation is used for early stopping, checkpoint selection, hyperparameter tuning and the choice of the final model; each trained model is evaluated on the test set once, to report its scores. The training loader drops its last incomplete batch, so 4,736 images (74 batches of 64) are seen per epoch.

4. Data Understanding and EDA

All statistics below were computed by `python -m src.cli eda` on the real train and test splits.

4.1 Sample images



Figure 1. Eight of the 200 species with one cleaned caption each (the same kind of sample as Figure 1 of the project specification).

The photographs show birds in the wild, in very different poses, sizes, lighting and backgrounds (water, branches, foliage, ground). Often the bird covers a small part of the frame (the White Pelican and the Black-footed Albatross), while others are close-ups (the Black-capped Vireo). The captions describe colours and body parts ("black beak and red eyering"), which is the information a captioning model must learn in Milestone 2. They are not guaranteed to be correct or complete: the caption for the Prairie Warbler calls it a "sparrow like bird".

4.2 Class distribution

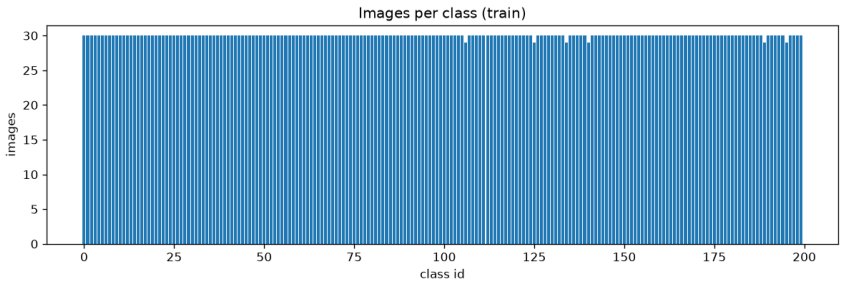


Figure 2. Images per class in the train split.

The training set is almost perfectly balanced: every class has 30 images, with a handful at 29. There is therefore no need for resampling or class weights, and macro-averaged and overall accuracy are nearly the same number. The difficulty comes from having few images per class, not from imbalance.

4.3 Image sizes and quality

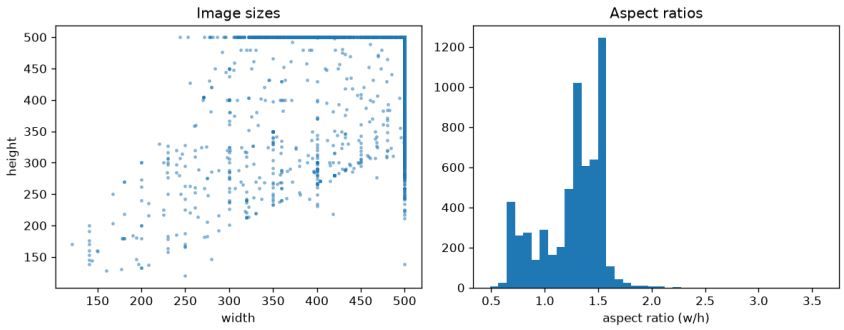


Figure 3. Left: image width and height. Right: aspect ratio (width/height).

Images are at most 500 px on the longer side; a large share are exactly 500 px wide or high, and smaller images are scattered below. Aspect ratios range from 0.5 (tall) to about 2.5, with most images between 1.2 and 1.6 (landscape). Because the images are not square, they cannot simply be stretched to 224x224 without distorting the bird, so the pipeline resizes first and then crops (Section 5).

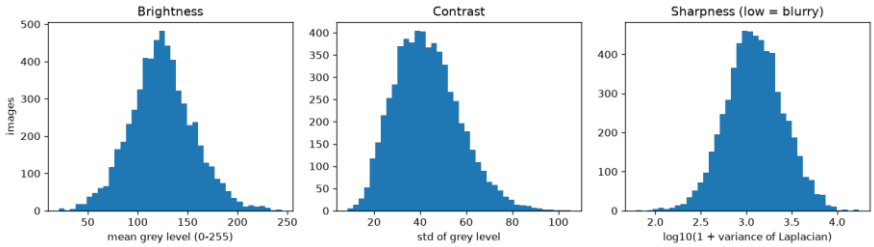


Figure 4. Brightness, contrast and sharpness of the 5,994 train images (measured on a 128x128 thumbnail so that scores are comparable across image sizes).

Image quality was measured per image: **brightness** (mean grey level), **contrast** (standard deviation of the grey level) and **sharpness** (variance of the Laplacian, a standard blur measure: low values mean blurry). All three are roughly bell-shaped, so there is no separate group of bad images. The median brightness is 123 of 255 (range 21 to 244), the median contrast is 42, and the sharpness spans more than two orders of magnitude (median 1,227; the blurriest 1% are below 223). Extremes exist but are few: 15 images are very dark (mean grey level below 40), 41 are very bright (above 215), and 46 have a side shorter than 200 px, so they are upscaled to 224. We kept them: they are real photographs of the species, and the random-resized-crop and colour-jitter augmentation already exposes the model to such variation.

The mean RGB of the dataset is (0.486, 0.499, 0.432) with standard deviation (0.226, 0.221, 0.260), close to the ImageNet statistics (0.485, 0.456, 0.406) and (0.229, 0.224, 0.225). The slightly higher green and blue come from the many water and foliage backgrounds. Because

they are close, ImageNet normalisation is appropriate for the pretrained models and costs nothing for the scratch model (Section 5.2).

- **Non-RGB files:** 4 of 5,994 train images are not 3-channel RGB (grayscale or alpha). They are converted to RGB when loaded.
- **Near-duplicates:** 2 pairs of train images have an identical perceptual hash. Two pairs out of 5,994 images is negligible, so they were kept and noted.
- **Corrupted files:** none; every image decoded when the statistics were computed.

4.4 Caption statistics

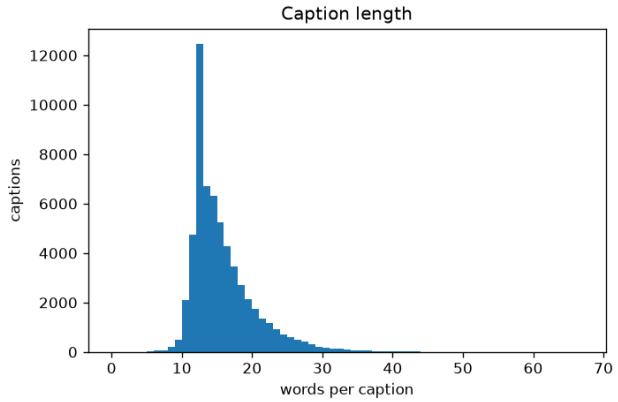


Figure 5. Caption length in words (all train captions, after cleaning).

Table 3. Caption statistics before and after cleaning (Section 5.1).

Statistic	Train, raw (lower-cased)	Train, cleaned	Test, cleaned
Images	5,994	5,994	5,794
Images without exactly 10 captions	0	0	0
Mean caption length (words)	15.24	15.28	15.26
Captions of at most 3 words	28	2	1
Captions of 40 words or more	124	128	117
Vocabulary size (distinct words)	6,690	3,738	3,841
Words that occur exactly once	3,487 (52%)	1,592 (43%)	1,720 (45%)
Duplicate captions within one image	639	646	673

Every image has exactly ten captions. Lengths peak at about 12 words with a long tail up to about 65 words, so a captioning decoder needs a maximum length around 30 to 40 tokens; 128 train captions (0.2%) have 40 words or more. **Encoding damage:** 27 train and 29 test captions have every space replaced by two Unicode replacement characters (U+FFFD), for example `this??bird??has??a??black...`, so the whole caption looks like a single word. That is why 28 raw train captions seem to have at most three words; 26 of them are these

damaged captions, and the punctuation step of the cleaning turns the replacement characters back into spaces. After cleaning only 2 very short captions remain (the shortest has one word, `this`). They are kept: they are legitimate, if poor, captions.

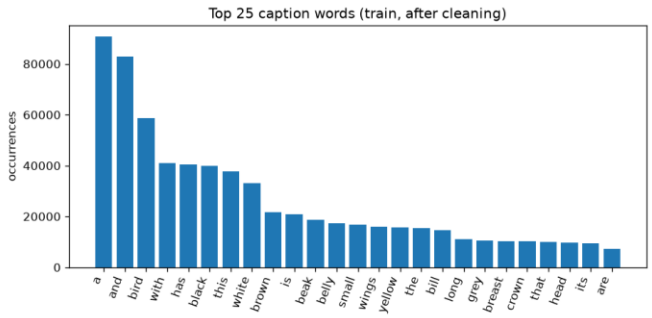


Figure 6. The 25 most frequent words in the cleaned train captions.

The most frequent words are function words and the vocabulary of plumage (colours, "bill", "wings", "crown", "breast"). **Noise in the raw text:** the vocabulary has 6,690 distinct tokens, but more than half of them (3,487) occur once. Most of this is not rare bird vocabulary but punctuation glued to words (`massive,`, `strip;`, `beak."`) and misspellings (`agrayish`, `winspan`, `magestic`, `eyrings`). Cleaning (Section 5.1) removes the punctuation variants, which brings the vocabulary down to 4,061 words, and the typo correction brings it to 3,738.

4.5 Summary of anomalies

Table 4. Anomalies found during EDA and what was done about them.

Finding	Size	Action
Class imbalance	none (30 per class)	no resampling
Non-RGB images	4 train images	convert to RGB in the transforms
Near-duplicate images (same perceptual hash)	2 train pairs	kept, documented
Very dark / very bright / small images	15 / 41 / 46 train images	kept; covered by augmentation
Duplicate captions in one image	639 train / 673 test (cleaned)	kept (harmless for training); note for M2
Punctuation glued to words	2,629 extra vocabulary entries	removed in the caption cleaning
Encoding damage (spaces replaced by U+FFFD)	27 train / 29 test captions	repaired by the cleaning (non-alphanumeric to space)
Misspelled words	323 words corrected in 381 captions	corrected (Section 5.1)
Very short / very long captions	2 / 128 train captions	kept, counted
Variable image size and aspect ratio	all images	resize 256, crop 224

5. Data Cleaning, Preprocessing and Augmentation

5.1 Caption cleaning

Captions are not used by the classifier in Milestone 1, but the cleaning is part of the data task and was done now, with the results saved as `outputs/data/captions_clean.json`, so that Milestone 2 starts from clean text. Steps (all in `src/data/captions.py`):

- **Split** `description` on the newline character into the ten individual captions and drop blank lines (for example from a trailing newline).
- **Unicode normalisation (NFKC) and lower-casing**, so that `Bird` and `bird`, or typographic and plain quotes, are one token.
- **Remove punctuation.** Apostrophes are deleted (`bird's` becomes `birds`, `isn't` becomes `isnt`); every other non-alphanumeric character (comma, full stop, semicolon, quotation mark, hyphen) is replaced by a space and whitespace is collapsed. Hyphens split compound words (`long-tailed` becomes `long tailed`), which matches how the CUB species names are written and avoids a separate vocabulary entry for every hyphenated variant. This removed about 2,600 vocabulary entries that were only a word plus punctuation.
- **Correct typos conservatively.** A word that occurs at most twice in the train captions, has at least six letters, and is exactly one edit (deletion, insertion, substitution or swap of two adjacent letters) away from a word that occurs at least 20 times is replaced by that frequent word (the most frequent one if there are several). The correction table is built from the **train** captions only, so no test words influence it, and applied to train and test. It corrects 323 words in 381 of the 59,940 train captions, for example `agrayish` to `grayish`, `winspan` to `wingspan`, `loarge` to `large`, `irange` to `orange`, `feathets` to `feathers` and `metalic` to `metallic`.
- **Known limits of the typo step.** It is a heuristic without a dictionary, so it can also change a valid but rare word into a frequent neighbour, for example `curling` to `curving` or `clunky` to `chunky`; in the first 20 corrections we count four or five such cases, so its precision is roughly 75-80%, and the changes touch only 0.6% of the captions. We accepted that trade-off because the misspellings it removes would otherwise each become a separate, almost never seen vocabulary entry. Words shorter than six letters are never changed because valid short words (`wind`, `bread`, `alone`) are often one letter from a frequent one.

- **Kept on purpose:** duplicate captions within an image (harmless), and very short or long captions (counted and reported in Table 2, not removed, so that every image keeps exactly ten captions).

5.2 Image preprocessing

- Convert every image to 3-channel RGB.
- Evaluation: resize the shorter side to 256, centre-crop 224x224. This keeps the full bird in view without stretching it.
- Normalise with the ImageNet mean and standard deviation for **every** encoder, so that the from-scratch and pretrained models see identical inputs and the comparison is fair, and so the pretrained weights get the statistics they were trained with.

5.3 Augmentation strategy

With only about 30 images per class, overfitting is the main risk, so training images are augmented on the fly. Each transform was chosen for birds specifically:

Table 5. Training augmentation.

Transform	Setting	Why
Resize + RandomResizedCrop	resize 256, crop 224, scale 0.5-1.0	bird appears at different scales/positions; scale floor 0.5 avoids crops that lose the bird
Horizontal flip	p = 0.5	a bird facing left or right is the same species
Rotation	+/- 15 degrees	tilted camera and flying/perched poses
Colour jitter	brightness, contrast, saturation 0.2; hue 0.02	lighting changes; hue kept tiny because plumage colour identifies species
No vertical flip	-	birds are photographed upright; upside-down images never occur

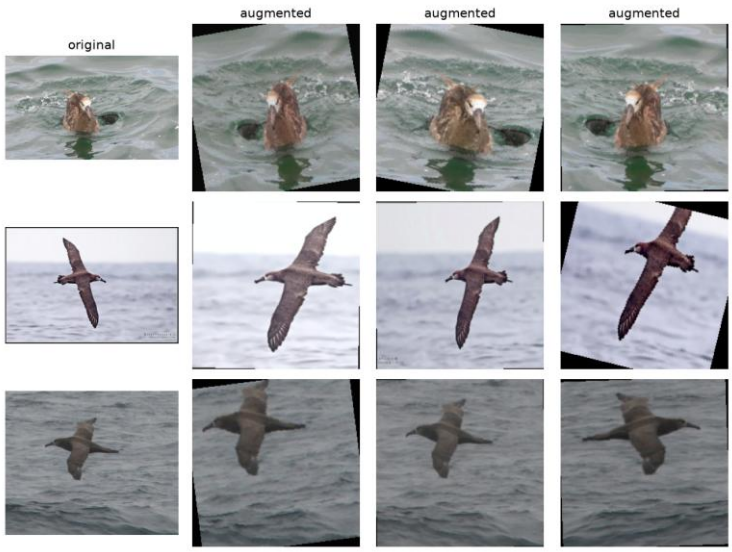


Figure 7. Originals (left column) and three random augmentations of each.

5.4 Colorization data (planned for Milestone 2)

Colorization pairs will be built from the same images: the grayscale version is the input and the original colour image is the target. Colour is therefore kept mostly intact in the classification augmentations, and the same train/validation/test split will be reused so that no test image leaks into training.

6. Models

Every model is a `Classifier`: an **encoder** that maps an image to a feature vector, followed by a head `Dropout -> Linear(feature_dim, 200)`. The encoder sits behind a small interface (`feature_dim`, `forward`, `gradcam_layer`) and is created by name through a registry. Swapping one encoder for another therefore changes one line of a config and nothing in the training loop. This is the **Factory/Registry** pattern. Other patterns used: **Strategy** (samplers for the tuner, trackers for logging) and **Template Method** (`Trainer.fit` fixes the epoch loop, early stopping and checkpointing).

6.1 ScratchCNN (baseline trained from scratch)

Five blocks of Conv 3x3 - BatchNorm - ReLU - MaxPool 2x2 with widths 32, 64, 128, 256 and 512, followed by global average pooling. It has about 1.6 million parameters and produces a 512-dimensional feature. A 224x224 input is reduced to 7x7 before pooling. It is deliberately small so it can be trained from random initialisation on 4,800 images in reasonable time. (Whether the specification allows a standard architecture as the "own CNN" is unconfirmed; we therefore also trained ResNet-18 from random initialisation.)

6.2 Pretrained encoders

Table 6. Encoders used.

Encoder	Parameters (approx.)	Feature size	Pretrained weights	Grad-CAM layer
ScratchCNN	1.6 M	512	none	last conv
ResNet-18	11.2 M	512	ImageNet-1k (V1)	layer4
ResNet-50	23.5 M	2048	ImageNet-1k (V2)	layer4
EfficientNet-B0	4.0 M	1280	ImageNet-1k (V1)	last features block

The original ImageNet classifier of each network is removed; its global-pooled feature goes to our 200-class head.

6.3 Training variants

- **From scratch:** random initialisation (ScratchCNN, and ResNet-18 as a second from-scratch baseline).
- **Frozen (feature extractor):** pretrained encoder with weights fixed and kept in evaluation mode (so BatchNorm statistics and dropout do not change); only the head is trained.
- **Fine-tuned:** all layers are trained, with a small learning rate for the pretrained backbone (1e-4) and a 10x larger one for the new head (1e-3). Small steps protect the pretrained features; the random head needs larger ones.

This gives nine final models: ScratchCNN; ResNet-18 from scratch, frozen, fine-tuned and tuned; ResNet-50 frozen and fine-tuned; EfficientNet-B0 frozen and fine-tuned.

7. Experiments

7.1 Training setup

Table 7. Common training settings.

Setting	Value
Optimiser	AdamW, two parameter groups (backbone / head), weight decay 1e-4
Learning-rate schedule	1 epoch linear warm-up, then cosine decay to about 0 (updated every step)
Loss	Cross-entropy with label smoothing 0.1
Batch size / epochs	64 / up to 30
Early stopping	stop after 7 epochs without improvement of validation accuracy; keep the best checkpoint
Regularisation	dropout 0.5 before the head; the augmentations of Section 5.3
Precision / hardware	mixed precision (fp16) on a free Colab T4 GPU

Setting	Value
Seed	42 (Python, NumPy, PyTorch, CUDA, deterministic cuDNN, seeded data-loader workers)
Speed	about 53 s per epoch for ResNet-18 (derived from the tuning timings, validation included)

Table 8. The nine final runs and their settings.

Run	Initialisation	Trained layers	Backbone lr	Head lr	Dropout
scratch_cnn	random	all	1e-3	1e-3	0.5
resnet18_random	random	all	1e-3	1e-3	0.5
resnet18_frozen	ImageNet	head only	-	1e-3	0.5
resnet18_finetune	ImageNet	all	1e-4	1e-3	0.5
resnet18_tuned	ImageNet	all	3e-4	3e-3	0.2
resnet50_frozen	ImageNet	head only	-	1e-3	0.5
resnet50_finetune	ImageNet	all	1e-4	1e-3	0.5
efficientnet_b0_frozen	ImageNet	head only	-	1e-3	0.5
efficientnet_b0_finetune	ImageNet	all	1e-4	1e-3	0.5

7.2 Results on the test set

All scores below are on the 5,794 test images, using the checkpoint with the best validation accuracy.

Table 9. Test results (precision, recall and F1 are macro averages).

Model	Top-1 acc. (%)	Top-5 acc. (%)	Precision (%)	Recall (%)	F1 (%)
ScratchCNN (from scratch)	17.31	42.91	16.68	17.47	15.47
ResNet-18 (from scratch)	37.73	68.93	37.58	38.03	36.65
ResNet-18 frozen	60.75	87.95	63.96	61.08	60.74
ResNet-18 fine-tuned	75.08	92.66	75.87	75.20	75.05
ResNet-18 tuned (grid best)	74.34	91.35	74.77	74.41	74.09
ResNet-50 frozen	63.44	89.52	65.31	63.76	63.11
ResNet-50 fine-tuned	**80.70**	**94.82**	**81.57**	**80.87**	**80.60**
EfficientNet-B0 frozen	76.94	94.32	78.25	77.07	76.98
EfficientNet-B0 fine-tuned	78.32	94.67	79.12	78.50	78.38

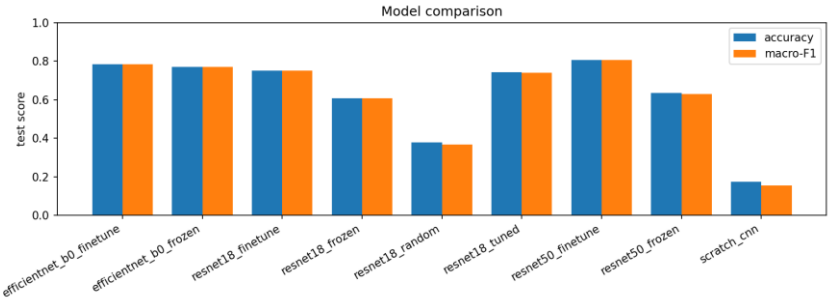


Figure 8. Test metrics of all models.

7.3 What the comparison shows

- **Transfer learning is the biggest effect.** ScratchCNN reaches 17.3%, ResNet-18 from random initialisation 37.7%, and the same ResNet-18 starting from ImageNet weights 75.1% when fine-tuned: about 37 points from pretraining alone. With 30 images per class, a network cannot learn good features from scratch, whereas ImageNet already contains many bird images.
- **Fine-tuning beats freezing** for every architecture: ResNet-18 +14.3 points (60.8 to 75.1), ResNet-50 +17.3 (63.4 to 80.7), EfficientNet-B0 +1.4 (76.9 to 78.3). ImageNet features are general; adapting the last layers to fine-grained bird details helps.
- **EfficientNet-B0 is the best frozen encoder** (76.9%) with only 4.0 M parameters, about a third of ResNet-18's. Its features transfer unusually well, which is why fine-tuning adds little there.
- **ResNet-50 fine-tuned is the best model** at 80.7% top-1 and 94.8% top-5, 2.4 points ahead of EfficientNet-B0 fine-tuned and 5.6 ahead of ResNet-18 fine-tuned. The larger model has more capacity for fine details.
- **Top-5 is much higher than top-1** (94.8% vs 80.7% for the best model): when the model is wrong, the correct species is usually among its first five guesses, because the errors are between look-alike species (Section 11).
- All fine-tuned models overfit to some degree: in the tuned ResNet-18 run the training loss falls to about 1.0 while the validation loss stays at about 2.0 (training curves in Section 9); an earlier ResNet-18 fine-tune with default settings reached 97% training accuracy against 73% validation accuracy.

8. Hyperparameter Tuning

Tuning is required in Milestone 1, so we compared **Grid Search** and **Random Search** on the same model: ResNet-18, pretrained and fine-tuned. Both methods search the same three hyperparameters (learning rate, weight decay, dropout), use the same budget, and are scored by **validation accuracy**. The head's learning rate is always 10x the backbone's. Because of free-GPU limits, the budget is small: **6 trials of 5 epochs each** per method.

Table 10. Search spaces.

Hyperparameter	Grid Search	Random Search
Learning rate (backbone)	{1e-4, 3e-4, 1e-3}	log-uniform in [1e-4, 1e-3]
Weight decay	{1e-4}	log-uniform in [1e-4, 1e-2]
Dropout	{0.2, 0.5}	uniform in [0.2, 0.5]
Trials	3 x 1 x 2 = 6	6 random draws

Table 11. Grid vs Random Search.

Method	Best validation acc.	Trials	Time (s)	Best setting
Grid Search	70.73%	6	1,578	lr 3e-4, wd 1e-4, dropout 0.2
Random Search	69.64%	6	1,605	lr 2.8e-4, wd 5.5e-4, dropout 0.48

Table 12. Grid Search: validation accuracy per trial (weight decay 1e-4).

lr	dropout 0.2	dropout 0.5
1e-4	63.30%	60.47%
3e-4	70.73%	70.73%
1e-3	65.89%	62.89%

Table 13. Random Search: the six sampled trials.

Trial	lr	weight decay	dropout	Val. acc.
0	5.9e-4	7.5e-4	0.46	67.97%
1	5.0e-4	1.5e-4	0.49	68.06%
2	5.8e-4	3.7e-3	0.24	68.31%
3	2.8e-4	5.5e-4	0.48	**69.64%**
4	4.4e-4	4.4e-3	0.33	68.06%
5	1.7e-4	1.3e-3	0.22	67.97%

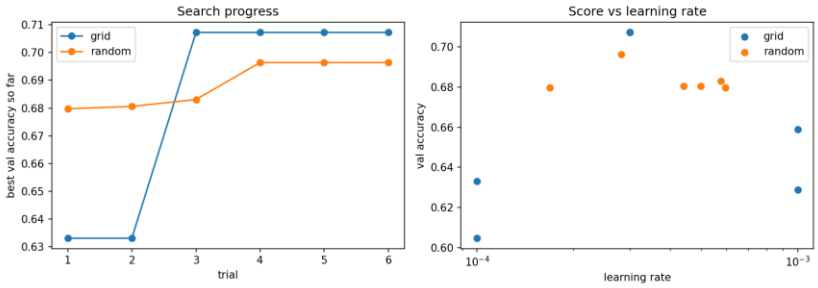


Figure 9. Grid vs Random Search: best score, trials and time.

8.1 Interpretation

- **Grid Search found the better setting** (70.7% vs 69.6%), and the cost was the same: about 1,580 s vs 1,605 s, because tuning time is dominated by training, not by how the points are chosen.
- **Why grid won here:** the grid included the extremes of the learning-rate range (1e-4 and 1e-3) and showed clearly that they are worse (60-66%). With only six draws, Random Search sampled learning rates only between 1.7e-4 and 5.9e-4, so all its trials landed in the middle and scored within 2 points of each other (68.0-69.6%). It was robust but never explored the bad regions, and in six draws it never hit the exact good spot (3e-4 with dropout 0.2).
- **Learning rate matters most.** In the grid, moving the learning rate from 3e-4 to either extreme costs 4-10 points, while dropout changes nothing at 3e-4 and 2-3 points at the extremes. In Random Search, weight decay spanning a 30-fold range moved the score by about 2 points.
- **The 1.1-point gap is small.** It is 13 of 1,199 validation images, and Random Search's best trial used almost the same learning rate (2.8e-4 vs 3e-4). The result supports "Grid was slightly better here", not "Grid is better".
- The two best grid trials tie at exactly 70.73% (848 of 1,199 validation images): at 3e-4, dropout 0.2 and 0.5 are indistinguishable on this budget.
- In general Random Search is expected to win when many hyperparameters exist and only a few matter, because it tries more distinct values of each. With three hyperparameters and six trials that advantage cannot appear; this comparison supports Grid Search only for this small space.

8.2 Does tuning carry over to the full run?

We retrained the grid winner with the full 30-epoch budget (`resnet18_tuned`). It scored **74.34%** on the test set against **75.08%** for the untuned fine-tune with the default settings, a difference of about 43 test images, which is within the noise between two training runs. On the validation split the two runs are identical to the second decimal (both reach a best validation accuracy of 73.39%), so the 5-epoch search did not improve the final model on either split. A likely reason is that a higher learning rate (3e-4) wins a short race because it learns faster, but gives no advantage once a cosine schedule has 30 epochs to converge. We report this as a negative result in Section 15.

9. Experiment Tracking

Every real run is logged to **Weights & Biases** (project `pixels-to-language`, workspace `hla-ain-shams`). The tracker sits behind a small interface (so a null tracker is used for smoke tests and unit tests). Each run records:

- its full configuration (training settings and encoder);
- per-epoch training/validation loss and accuracy and the learning rate (the curves);
- the best checkpoint as a W&B artifact named `<run>-best`, which later let us restore the models after Colab lost its local files (Section 15).

Runs are grouped: `m1-runs` for the nine final runs, `tune-grid` and `tune-random` for the tuning trials (named `grid-trial-i` and `random-trial-i`). The project holds 38 runs in total; names repeat because interrupted Colab sessions were re-run. Three runs are named `resnet18_tuned`: a 1-second run that was killed, the first run (which used the default settings, see Section 15) and the retrained run whose checkpoint is evaluated in Section 7.



Figure 10. Weights & Biases: validation accuracy per epoch. The 5-epoch curves are the tuning trials; the two 30-epoch curves are the two resnet18_tuned runs.

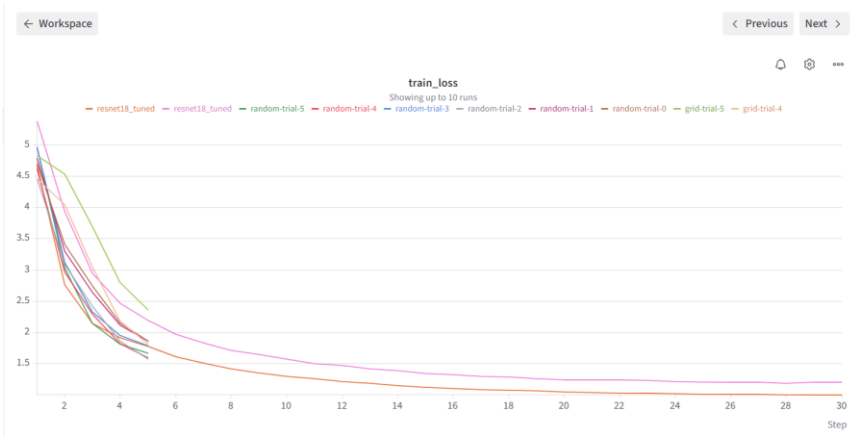


Figure 11. Weights & Biases: training loss per epoch for the same runs.

NAME 38 listed	STATE	NOTES	USER	TAGS	CREATED	RUNTIME	SWEEP
resnet18_tuned	Finished	Add notes...	hla-waleedm20(		19h ago	30m 6s	-
resnet18_tuned	Finished	Add notes...	hla-waleedm20(		3d ago	25m 54s	-
resnet18_tuned	Killed	Add notes...	hla-waleedm20(		3d ago	1s	-
random-trial-5	Finished	Add notes...	hla-waleedm20(		3d ago	4m 26s	-
random-trial-4	Finished	Add notes...	hla-waleedm20(		3d ago	4m 27s	-
random-trial-3	Finished	Add notes...	hla-waleedm20(		3d ago	4m 25s	-
random-trial-2	Finished	Add notes...	hla-waleedm20(		3d ago	4m 27s	-
random-trial-1	Finished	Add notes...	hla-waleedm20(		3d ago	4m 27s	-
random-trial-0	Finished	Add notes...	hla-waleedm20(		3d ago	4m 27s	-

1-20 of 38

Figure 12. Weights & Biases runs table (state, creation time and runtime). One 1-second resnet18_tuned run was killed and is not used.

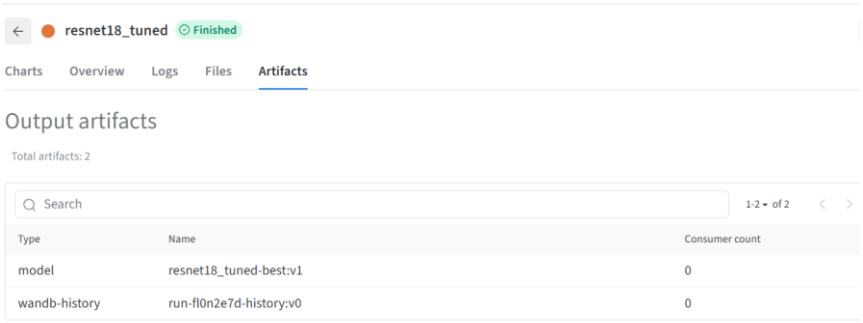


Figure 13. Weights & Biases Artifacts tab of the retrained resnet18_tuned run: the best checkpoint is stored as the model artifact resnet18_tuned-best:v1.

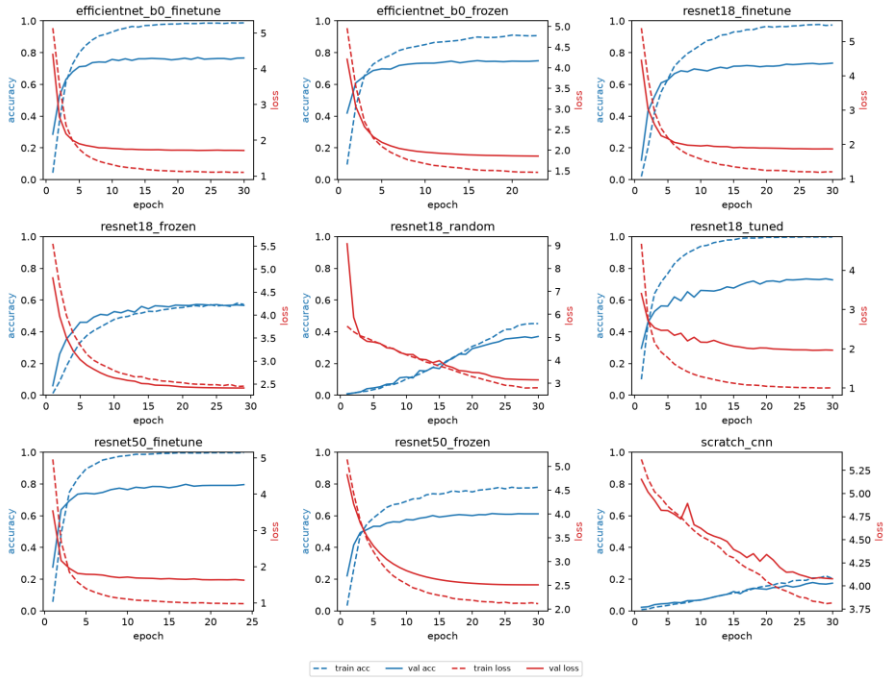


Figure 14. Training (dashed) and validation (solid) accuracy and loss of each of the nine final runs, rebuilt from the Weights & Biases histories with `fetch-history`.

Reading the curves. (1) The two from-scratch models are **not overfitting; they are under-trained.** ScratchCNN reaches only about 18% validation accuracy, and its training and validation curves are almost the same and still rising at epoch 30, so the 30-epoch limit stopped it before it converged. ResNet-18 from random weights is still improving at epoch 30 as well (about 45% training and 37% validation accuracy). Their test scores are therefore lower bounds. (2) The frozen encoders level off after roughly 15 to 20 epochs because only the linear head learns; ResNet-18 frozen stays at about 57% on both training and validation, while ResNet-50 frozen (78% train, 61% validation) and EfficientNet-B0 frozen (90% train, 75% validation) show a gap. (3) The fine-tuned models fit the training set almost perfectly

(ResNet-50 reaches nearly 100% training accuracy by about epoch 10) while validation accuracy flattens (ResNet-50 at about 79%, EfficientNet-B0 at about 77%, ResNet-18 at about 73%). Validation loss stays flat instead of rising, so this is memorisation of the small training set without the validation score getting worse; label smoothing and the cosine schedule probably help here. Early stopping ended ResNet-50 fine-tuned at epoch 24, EfficientNet-B0 frozen at 23 and ResNet-18 frozen at 29; the other six ran all 30 epochs.

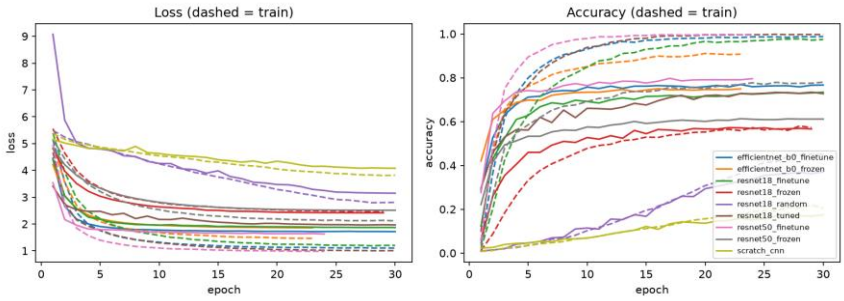


Figure 15. All nine runs on one plot: validation (solid) and training (dashed) loss, and accuracy.

10. Evaluation Metrics

Classification is judged on the test set with the metrics required by the specification, each computed with scikit-learn. With 200 classes of equal size, macro and overall values are close, but the macro versions are reported so that no class is hidden by the others.

Table 14. Classification metrics.

Metric	What it measures	How to read it
Accuracy (top-1)	share of test images whose predicted species is exactly right	80.7% means 4 in 5 birds are named correctly; chance is 0.5%
Top-5 accuracy	share of images whose true species is among the model's five highest scores	forgiving for look-alikes; a big gap to top-1 means errors are between similar species
Precision (macro)	of the images predicted as a species, the share that really belong to it, averaged over species	low precision = the model over-predicts that class (false alarms)
Recall (macro)	of the images that truly belong to a species, the share found, averaged over species	low recall = the species is often missed or confused with another
F1 (macro)	harmonic mean of precision and recall per species, then averaged	single balanced score; punishes a model that is good at precision but poor at recall
Confusion matrix	counts of true species (rows) vs predicted species (columns)	off-diagonal cells show which species are mixed up; used to list the most confused pairs

11. Results and Discussion

Best model: ResNet-50 fine-tuned, 80.70% accuracy, 94.82% top-5 and 80.60% macro F1. Precision (81.6%) and recall (80.9%) are close to each other, so the model is not biased toward over- or under-predicting particular species.

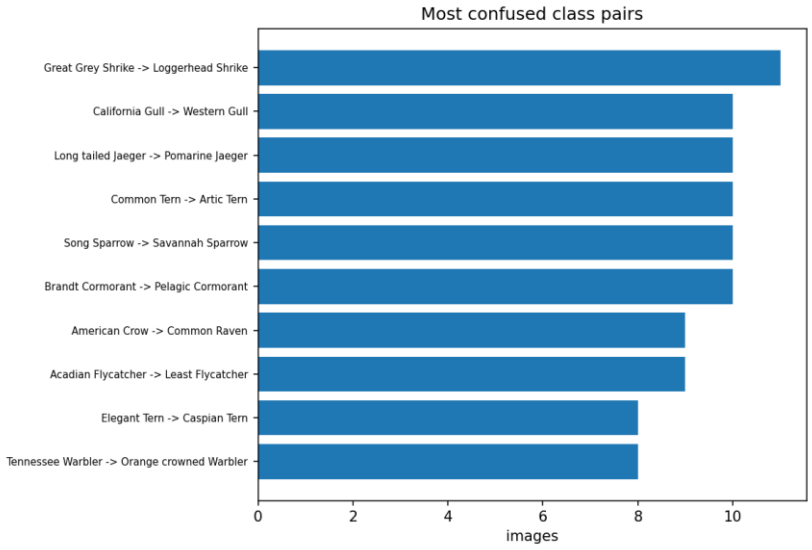


Figure 16. The ten most frequent mistakes of ResNet-50 fine-tuned (true species -> predicted species).

Where the errors are. The most frequent confusions are between species that even a human birder finds hard to separate: Great Grey Shrike and Loggerhead Shrike (11 images), California Gull and Western Gull (10), Long-tailed Jaeger and Pomarine Jaeger (10), Common Tern and Arctic Tern (10), Song Sparrow and Savannah Sparrow (10), Brandt Cormorant and Pelagic Cormorant (10). Terns, gulls, jaegers, shrikes, sparrows and cormorants are groups of close relatives that differ in small marks such as bill colour or wing pattern. The same pairs recur across models (for example Elegant Tern and Caspian Tern, American Crow and Common Raven), which suggests the limit is the information available at 224x224 and with 30 images per class, not one model's weakness.

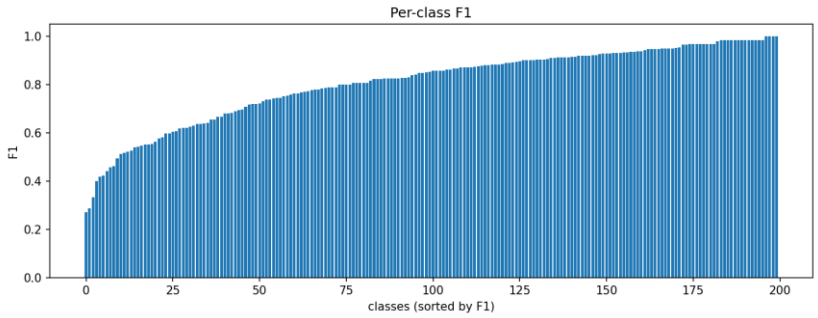


Figure 17. Per-class F1 of ResNet-50 fine-tuned.

Overfitting. The gap between training and validation shows that the models memorise the small training set. In the retrained (tuned) ResNet-18 the training loss reaches about 1.0 and the validation loss plateaus near 2.0, and in the first ResNet-18 fine-tune (default settings) training accuracy reached 97% while validation accuracy stayed at 73%. The tuned run, with its higher learning rate and lower dropout, fits the training set even more closely (lower training loss in the Weights & Biases curves) without a better validation score, which is consistent with its slightly lower test accuracy. Stronger augmentation, more regularisation or larger inputs are the natural next steps. (The loss includes label smoothing 0.1, so it does not approach zero; compare it across runs, not in absolute terms.)

From scratch vs pretrained. The scratch CNN's 17% shows what 4,800 images can teach a small network, and the confusions of the scratch model include unrelated species (Frigatebird predicted as Pomarine Jaeger, Horned Lark as Geococcyx), whereas the top confusions of the fine-tuned models are all between related species. Pretraining therefore improves both the accuracy and the quality of the errors.

12. Explainability: Grad-CAM

Grad-CAM shows which image regions pushed the network toward a class. For a chosen class, the gradients of its score are averaged over each channel of the last convolutional layer to give one weight per channel; the weighted sum of the channel maps is passed through a ReLU, resized to the image and drawn as a heat map (red = most influential) over the photo. We hook `layer4` of the ResNets, the last conv layer of ScratchCNN, and the last feature block of EfficientNet-B0.

For each model, four sets of four test images are shown, chosen automatically by `evaluate`:

- **confident correct:** right predictions with the highest probability;
- **barely correct:** right predictions with the lowest probability;
- **confident wrong:** wrong predictions with the highest probability;
- **confused pair:** wrong predictions of the single most frequent mistake.

12.1 ResNet-50 fine-tuned (best model)

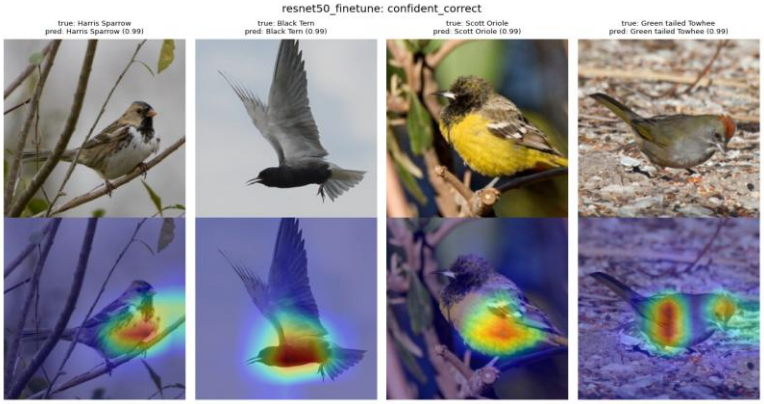


Figure 18. ResNet-50 fine-tuned, confident correct predictions (probability 0.99).



Figure 19. ResNet-50 fine-tuned, barely correct predictions (probability 0.06-0.07).

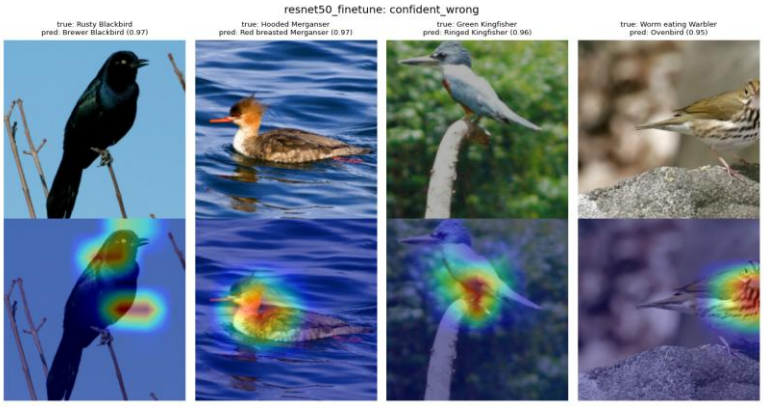


Figure 20. ResNet-50 fine-tuned, confident wrong predictions (probability 0.95-0.97).

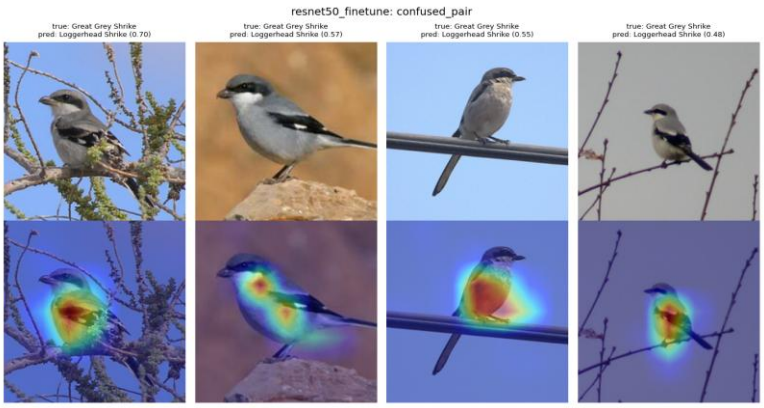


Figure 21. ResNet-50 fine-tuned, the most frequent mistake: Great Grey Shrike predicted as Loggerhead Shrike.

Confident correct. The heat sits on the bird's body in all four images: the streaked chest and belly of the Harris's Sparrow, the body and head of the Black Tern, the yellow breast of the Scott Oriole, and the back and head of the Green-tailed Towhee. Branches, sky and gravel are ignored. The model is using plumage colour and pattern on the body, which is also how people tell these species apart, so the high confidence is backed by sensible evidence.

Barely correct. These are right, but with a probability of only 6-7%. In three of the four images the hottest region is **not on the bird**: foliage in the top corner (Bohemian Waxwing), an empty lower-left corner (Herring Gull) and leaves at the top left (Fox Sparrow). Only the Fish Crow map falls on the bird (its head). These are the hardest correct test images (low contrast, unusual crops or poses), and the maps suggest that when the bird itself gives weak evidence the model falls back on context. Such predictions are fragile: they are right for weak or partly irrelevant reasons.

Confident wrong. In most cases the model is looking at the bird, so the mistake is not caused by the background: for the Hooded Merganser predicted as Red-breasted Merganser the heat is on the head and chest, and for the Worm-eating Warbler predicted as Ovenbird it is on the streaked breast (the bird is cut off at the image edge, and both species have streaked breasts). For the Rusty Blackbird predicted as Brewer's Blackbird, part of the heat falls on an empty area beside the body, and both are uniformly dark birds. The Green Kingfisher predicted as Ringed Kingfisher has heat on the chest; the photo is ambiguous to us, so an atypical or mislabelled image cannot be ruled out. The errors are therefore between near-identical plumages, which agrees with the confusion pairs in Section 11. The 0.95-0.97 confidence also shows the model is overconfident when it is wrong.

Confused pair. For the Great Grey Shrike / Loggerhead Shrike mistake the confidence is lower (0.48-0.70) and the heat covers the body and wing patch, in one image the head and wing. The two species differ mainly in the width of the black face mask and the bill, which

are only a few pixels in these photos (in two of them the bird fills a small part of the frame). Our reading is that at 224 px the model cannot resolve that cue and relies on the shared grey-and-black body pattern; higher-resolution inputs would test this.

12.2 ScratchCNN (baseline trained from scratch)



Figure 22. ScratchCNN, confident correct predictions (probability 0.83-0.94).



Figure 23. ScratchCNN, barely correct predictions (probability 0.02-0.03).

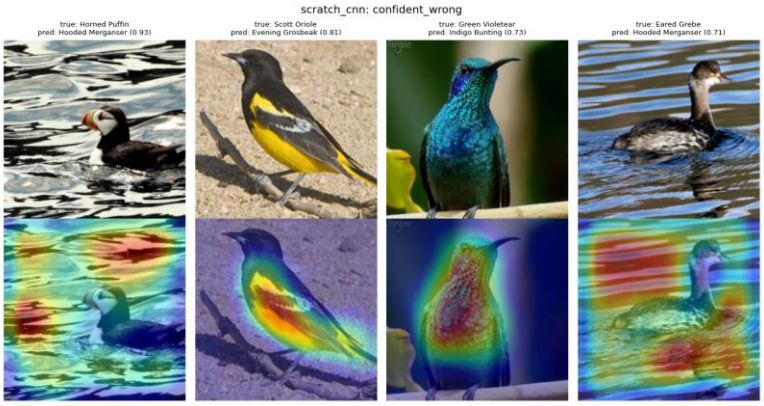


Figure 24. ScratchCNN, confident wrong predictions (probability 0.71-0.93).



Figure 25. ScratchCNN, the most frequent mistake: Frigatebird predicted as Pomarine Jaeger.

Confident correct. The maps fall on high-contrast colour patches: the red head and black-and-white back of the Red-bellied Woodpecker and the striped flank of the Hooded Merganser. In the first merganser image part of the heat is on the **water reflection**, which shows the network is responding to colour and texture blobs rather than to the bird as a whole.

Barely correct. With probabilities of 2-3%, the maps point at branches, ground and leaves in three of four images (the lower-right branch tip for the Spotted Catbird, the ground for the White-crowned Sparrow, leaves for the Warbling Vireo). These are essentially lucky guesses.

Confident wrong. Here the failure mode is clear: for the Horned Puffin and the Eared Grebe, both predicted as Hooded Merganser, the heat is on the **rippling water**, not on the bird. The network has learned the shortcut "water scene -> merganser". The other two errors follow colour alone: the Scott Oriole's yellow-and-black body is read as Evening Grosbeak, and the Green Violetear's blue-green plumage as Indigo Bunting.

Confused pair. Frigatebirds are predicted as Pomarine Jaegers with low confidence (0.16-0.20). The heat lies on the dark wings against the sky; the forked tail and white throat patch that identify a frigatebird are not used. The faint rectangular frame visible in several maps is a border effect of zero padding in this small network, not image content.

12.3 What the two models tell us

- **ResNet-50 looks at the bird, ScratchCNN at colour and scenery.** The pretrained model concentrates on the body and plumage and ignores the background when it is confident, and its errors are between near-identical species. The scratch model uses colour blobs and water or ground texture, so its errors are between unrelated species (puffin and grebe called merganser). This is consistent with the scores (80.7% vs

17.3%) and with the idea that ImageNet pretraining provides object-level features that 4,800 images cannot teach.

- **Low-confidence correct predictions are the weak spot of both models.** In both, the maps for the barely correct group often leave the bird, so accuracy overstates how well the evidence supports some predictions.
- **Limits of this analysis.** Grad-CAM shows where evidence comes from, not why a feature matters. Each group is four images selected automatically by confidence, so these are illustrations, not statistics, and our reading of species identity in the photos is that of non-experts.

13. Benchmarking

We compare against published top-1 accuracies on the same CUB-200-2011 test set (5,794 images) and the same metric (top-1 accuracy). The published numbers were read from the cited papers; their setups differ from ours, which is explained below.

Table 15. Our results vs published results.

Model	Top-1 (%)	Setup
Ours: ResNet-50 fine-tuned	**80.70**	ImageNet-pretrained, 224 px, 4,795 train images, 30 epochs max
Ours: EfficientNet-B0 fine-tuned	78.32	ImageNet-pretrained, 224 px
Ours: ResNet-18 fine-tuned	75.08	ImageNet-pretrained, 224 px
ResNet-50 baseline (NTS-Net paper, Table 4)	84.5	ImageNet-pretrained, 448 px, full 5,994 train images
NTS-Net (K=4)	87.5	ResNet-50 + navigator/teacher/scrutinizer, 448 px
ViT-B/16 baseline (TransFG paper, Table 1)	90.3	ImageNet-21k pretrained, 448 px
TransFG	91.7	ViT-B/16 + part-selection module, 448 px

Reading the gap. Our best model is 3.8 points below the plain ResNet-50 baseline of the NTS-Net paper, which uses the same architecture family, and 11 points below TransFG. Likely causes, roughly in order of importance:

- **Input resolution:** the published models use 448x448 inputs; ours use 224x224. Fine-grained cues (bill shape, eye ring, wing bars) are only a few pixels wide at 224 px. This is probably the largest factor, and it is also what keeps the confusions between close relatives high.
- **Training data:** we held out 20% of the train split for validation, so models see 4,795 instead of 5,994 images; the published baselines train on the full train split.

- **Short, untuned schedule:** at most 30 epochs with early stopping, default hyperparameters for ResNet-50, and a free-GPU time budget; the papers train longer with tuned schedules.
- **Method:** NTS-Net and TransFG add part-localisation modules (attention to the discriminative regions) and TransFG uses a transformer pretrained on ImageNet-21k, a much larger pretraining set. These are architectural advantages we did not try.
- **Comparability:** we compare top-1 accuracy on the official test split, which is the same for all rows. Differences in pretraining data and resolution remain, so the comparison shows where we stand, not that the methods are equal.

The ResNet-50 baseline is the fairest reference: the same architecture, the same test set and metric. We are within about 4 points of it despite using half the resolution and less training data, which suggests our pipeline is sound.

14. Final Model Selection

Selected for classification: ResNet-50, fine-tuned (resnet50_finetune). The choice is made on the **validation** split, not the test set. Table below ranks every run by its best validation accuracy; the test accuracy is shown next to it for reporting only.

Table 16. Model selection: validation accuracy decides, test accuracy is only reported.

Run	Best val. acc. (%)	Epochs run	Test acc. (%)
resnet50_finetune	**79.73**	**24**	**80.70**
efficientnet_b0_finetune	76.90	30	78.32
efficientnet_b0_frozen	74.98	23	76.94
resnet18_finetune	73.39	30	75.08
resnet18_tuned	73.39	30	74.34
resnet50_frozen	61.30	30	63.44
resnet18_frozen	57.21	29	60.75
resnet18_random	37.03	30	37.73
scratch_cnn	17.85	30	17.31

ResNet-50 fine-tuned is first on validation and also first on test, so the choice does not depend on which split is used. It also has the highest top-5 accuracy (94.82%) and macro F1 (80.60%), with balanced precision and recall.

Table 17. Final candidates.

Candidate	Test acc. (%)	Parameters	Comment
ResNet-50 fine-tuned	80.70	23.5 M	best accuracy; selected

Candidate	Test acc. (%)	Parameters	Comment
EfficientNet-B0 fine-tuned	78.32	4.0 M	2.4 points lower with 6x fewer parameters; best light alternative
EfficientNet-B0 frozen	76.94	4.0 M (head only trained)	best result with no encoder training
ResNet-18 fine-tuned	75.08	11.2 M	simple, fast baseline

For Milestone 2 the trained encoder becomes the backbone of the colorization network and the frozen encoder of the caption decoder. The ResNet-50 encoder (2048-dimensional features) is the accuracy-optimal choice; EfficientNet-B0 is the fallback if the colorization decoder or the captioning model does not fit the free-GPU memory and time limits. We will confirm the choice at the start of Milestone 2.

15. Failures and Limitations

This section records what did not work and why, as required by the specification.

15.1 What went wrong

- **Results lost on Colab (twice).** First, runs and tuning outputs written to the Colab runtime disk were lost when the daily GPU limit ended the session; the evaluate step then failed because the checkpoints no longer existed. Second, after we added Drive-backed output folders, the symlink step had not taken effect after a runtime reset, so the final runs again wrote to local disk and were erased by a page refresh (the `final` folder on Drive was empty and `ls -l outputs` showed no symlinks). **Fix:** the trainer now saves a resumable `.last.pt` every epoch, `--resume` skips finished runs, a `fetch-checkpoints` command restores the best checkpoints from W&B artifacts, and we now verify `ls -l outputs` before any training. All nine checkpoints were restored from W&B and re-evaluated. The per-epoch history files of seven runs were not restored with the checkpoints; they were rebuilt afterwards from the W&B histories with `fetch-history`.
- **Caption cleaning took two attempts.** The first version only lower-cased the captions, which left 6,690 vocabulary entries (half of them seen once), mostly punctuation glued to words, and left 27 train captions whose spaces were damaged by an encoding error looking like one-word captions. After stripping punctuation the vocabulary fell to 4,061. The first typo correction (words of five letters or more) also changed valid words (`alone` to `along`, `bread` to `broad`), so we raised the minimum to six letters (323 corrections instead of 530); some valid words are still changed (Section 5.1).

- **The final model was first chosen on test accuracy.** Our first comparison ranked the runs by test score, which lets the test set influence the decision. We now choose on validation accuracy and only report test accuracy (Section 14).
- **The "tuned" model was not tuned.** The first `resnet18_tuned` run gave test metrics identical to `resnet18_finetune` to four decimals. The config still held the default hyperparameters because the grid-search winner had never been copied into it, so the same seed produced the same model. We noticed the identical numbers, fixed the config (lr 3e-4, head lr 3e-3, dropout 0.2) and retrained.
- **Tuning did not help at the full budget.** The grid winner scored 74.34% against 75.08% for the untuned model (Section 8.2). The 5-epoch search favours faster learning rates and the budget (6 trials) was too small to separate Grid from Random Search.
- **The from-scratch models are weak.** ScratchCNN (17.3%) and ResNet-18 from random weights (37.7%) are far below the pretrained models. With about 30 images per class and a limited budget this is expected; we did not try to rescue them with longer training or heavier augmentation, so they are a lower bound for from-scratch training rather than a best effort. Their curves (Section 9) are still rising at epoch 30, so more epochs would help.

15.2 Limitations

- Inputs are 224x224; the published results use 448x448 (Section 13).
- The final models were trained on 80% of the train split; we did not retrain on train plus validation.
- Each result is a single run with seed 42, so differences below about 1 point (such as tuned vs untuned) are within noise; no repeated seeds or confidence intervals were computed.
- GPU training is not bit-for-bit reproducible across hardware or PyTorch builds.
- The caption typo correction is a dictionary-free heuristic with roughly 75-80% precision on the sample we checked; a few valid words were changed (Section 5.1). It was not evaluated on a labelled set of typos.
- The published benchmark numbers were checked against the cited papers (NTS-Net 84.5 and 87.5, TransFG 90.3 and 91.7), but their training setups differ from ours (Section 13).

15.3 What we would try next

- Train at 448x448 (or 320) with the best encoder; this is the most likely way to close the gap to the published baselines.
- Tune at a longer per-trial budget or with successive halving, and use more trials so that Random Search can show its advantage.
- Run several seeds for the final models and report mean and spread.
- Add stronger regularisation (mixup/cutmix, stochastic depth) against the visible overfitting.

16. Team Contributions

Team 7 split the project by task. Task 1 (this milestone) was carried out by Hla Waleed; the two Milestone 2 tasks are assigned to Malak Khaled and Kenzy Tarek, who will build on the same configuration system, tracker, CLI and data split.

Table 18. Team contributions.

Member	Student ID	Responsibility
Hla Waleed	23P0210	Task 1: EDA and data cleaning, classification and encoder training (scratch baseline and transfer learning), hyperparameter tuning, experiment tracking, Grad-CAM, benchmarking, and this report
Malak Khaled	2300447	Task 2 (Milestone 2): image colorization with the trained encoder as the backbone
Kenzy Tarek	2301141	Task 3 (Milestone 2): image captioning with the frozen encoder and an autoregressive decoder

17. Conclusion

Milestone 1 delivered a complete, reproducible classification pipeline for CUB-200-2011: an EDA that found a balanced but small and fine-grained dataset with variable image shapes and noisy captions; a cleaning and augmentation strategy suited to birds; nine trained models; a Grid vs Random Search comparison; Grad-CAM explanations; experiment tracking; and a benchmark against published results.

The main findings are that transfer learning matters more than anything else (17% from scratch vs 75% pretrained, same data), that fine-tuning beats frozen features, that **ResNet-50 fine-tuned is the best model at 80.7%**, and that the remaining errors are between close

relatives and are limited mainly by resolution and data size. The tuning experiment did not improve the final model, and we report that honestly together with the infrastructure failures.

Next (Milestone 2, due 18 December): colorization with the trained encoder as the backbone of a U-Net or encoder-decoder (grayscale input, original colour target, PSNR and MAE), and captioning with a frozen encoder and an autoregressive decoder (BLEU and ROUGE-L). The same split, config system, tracker and CLI will be reused.

18. References

[1] Wah, C., Branson, S., Welinder, P., Perona, P., Belongie, S. The Caltech-UCSD Birds-200-2011 Dataset. Technical Report CNS-TR-2011-001, Caltech, 2011.

[2] Hugging Face datasets `Multimodal-Fatima/CUB_train` and `Multimodal-Fatima/CUB_test` (CUB-200-2011 with captions).

[3] Yang, Z., Luo, T., Wang, D., Hu, Z., Gao, J., Wang, L. Learning to Navigate for Fine-grained Classification (NTS-Net). ECCV 2018. https://arxiv.org/abs/1809.00287

[4] He, J., Chen, J.-N., Liu, S., et al. TransFG: A Transformer Architecture for Fine-grained Recognition. AAAI 2022. https://arxiv.org/abs/2103.07976

[5] He, K., Zhang, X., Ren, S., Sun, J. Deep Residual Learning for Image Recognition. CVPR 2016. https://arxiv.org/abs/1512.03385

[6] Tan, M., Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019. https://arxiv.org/abs/1905.11946

[7] Selvaraju, R. R., et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017. https://arxiv.org/abs/1610.02391

[8] Loshchilov, I., Hutter, F. Decoupled Weight Decay Regularization (AdamW). ICLR 2019. https://arxiv.org/abs/1711.05101

[9] Loshchilov, I., Hutter, F. SGDR: Stochastic Gradient Descent with Warm Restarts (cosine schedule). ICLR 2017. https://arxiv.org/abs/1608.03983

[10] Szegedy, C., et al. Rethinking the Inception Architecture for Computer Vision (label smoothing). CVPR 2016. https://arxiv.org/abs/1512.00567

[11] Bergstra, J., Bengio, Y. Random Search for Hyper-Parameter Optimization. JMLR 13, 2012.

[12] PyTorch and torchvision (pretrained model weights); scikit-learn (metrics and splits); Hugging Face `datasets`; Weights & Biases (experiment tracking).

[13] Project specification: Deep Learning - Final Project (CSE 485/488), Ain Shams University.